

EXPERIMENT 2

Aim: Create a Blockchain using Python

Theory:

A blockchain is a distributed and tamper-evident digital ledger in which data is stored in a sequence of connected blocks. Each block contains data, a timestamp, a hash, and the hash of the previous block, which creates a secure link between blocks.

In this program:

1. **Block Creation** – A block is created with an index, timestamp, data, previous hash, and nonce.
2. **Proof-of-Work (PoW)** – A simple mining process is implemented by changing the nonce until the block hash starts with a specified number of zeros.
3. **Hashing** – The SHA-256 cryptographic hashing algorithm is used to generate a unique hash for each block.
4. **Blockchain Validation** – The `is_chain_valid()` function checks whether the hashes are correct and whether every block is properly connected to the previous block.

This program runs locally in Python and provides a simple demonstration of how blocks are created, mined, linked together, and validated. It is designed as an introductory model for understanding the basic working principles of blockchain technology.

1. What is a Blockchain?

A blockchain is a chain of blocks in which each block stores information and is connected to the previous block using its hash. This makes unauthorized modification of stored data easy to detect.

2. Process of Mining

Mining is the process of finding a suitable nonce that produces a hash satisfying the required condition. In this experiment, the mining process searches for a hash beginning with three zeros (000).

3. How to Check the Validity of Blocks in a Blockchain

The validity of the blockchain is checked using the `is_chain_valid()` function. It verifies that each block's hash is correct and that its `previous_hash` matches the hash of the preceding block. If any block is modified, the blockchain becomes invalid.

Code

1. Importing Libraries

```
import hashlib
import time
```

2. Creating the Block Class

```
# Define the structure of a block
class Block:

    def __init__(self, index, data, previous_hash):
        self.index = index
        self.timestamp = time.time()
        self.data = data
        self.previous_hash = previous_hash
        self.nonce = 0

        # Generate the initial hash
        self.hash = self.calculate_hash()

    # Calculate SHA-256 hash of the block
    def calculate_hash(self):
        block_data = (
            str(self.index) +
            str(self.timestamp) +
            str(self.data) +
            str(self.previous_hash) +
            str(self.nonce)
        )

        return hashlib.sha256(block_data.encode()).hexdigest()

# Define the blockchain
class Blockchain:

    def __init__(self):
        # Create the first block (Genesis Block)
        self.chain = [self.create_genesis_block()]

        # Set mining difficulty
        self.difficulty = 3
```

```
# Create the first block
def create_genesis_block(self):
    return Block(0, "Genesis Block", "0")

# Get the latest block
def get_latest_block(self):
    return self.chain[-1]
```

3. Adding a New Block

```
# Add a new block to the blockchain
def add_block(self, new_block):

    # Link the new block with the previous block
    new_block.previous_hash = self.get_latest_block().hash

    # Mine the block
    self.mine_block(new_block)

    # Add the block to the chain
    self.chain.append(new_block)
```

4. Implementing Proof-of-Work

```
# Mine a block using Proof-of-Work
def mine_block(self, block):

    # Find a hash starting with 000
    while not block.hash.startswith("0" * self.difficulty):

        # Change the nonce
        block.nonce += 1

        # Calculate a new hash
        block.hash = block.calculate_hash()

    print("Block mined!")
    print("Hash:", block.hash)
```

5. Creating the Blockchain

```
# Create an instance of the blockchain
my_blockchain = Blockchain()
print("Blockchain created!")
```

6. Adding Blocks to the Blockchain

```
# Create the first transaction block
block1 = Block(
    1,
    "Atharva sends 10 coins to Nidhi",
    ""
)
```

```
# Add block to the blockchain
my_blockchain.add_block(block1)
```

```
# Create the second transaction block
block2 = Block(
    2,
    "Nidhi sends 5 coins to Tanaya",
    ""
)
```

```
# Add block to the blockchain
my_blockchain.add_block(block2)
```

```
Block mined!
Hash: 00054c3d678dc3454a81d0159f4a19658a1b19682890d3d55dd769dd881ef6a3
Block mined!
Hash: 000d5d5deba5da8269be589dfbcb43e75bcb4625cb2df65a84b3bb3c1c6f17e7
```

7. Displaying the Blockchain

```
# Display all blocks in the blockchain
for block in my_blockchain.chain:

    print("\n-----")
    print("Block Number :", block.index)
    print("Data      :", block.data)
    print("Hash       :", block.hash)
    print("Previous Hash:", block.previous_hash)
```

```
print("Nonce      :", block.nonce)
```

```
-----
Block Number : 0
Data          : Genesis Block
Hash          : 8428e38918419eb3514e34ceb0d96975c874338d0f17fd393cf9b6cffe23ebb4
Previous Hash: 0
Nonce         : 0

-----

Block Number : 1
Data          : Atharva sends 10 coins to Nidhi
Hash          : 00054c3d678dc3454a81d0159f4a19658a1b19682890d3d55dd769dd881ef6a3
Previous Hash: 8428e38918419eb3514e34ceb0d96975c874338d0f17fd393cf9b6cffe23ebb4
Nonce         : 3684

-----

Block Number : 2
Data          : Nidhi sends 5 coins to Tanaya
Hash          : 000d5d5deba5da8269be589dfbcb43e75bcb4625cb2df65a84b3bb3c1c6f17e7
Previous Hash: 00054c3d678dc3454a81d0159f4a19658a1b19682890d3d55dd769dd881ef6a3
Nonce         : 3523
```

8. Checking Blockchain Validity

```
# Check the validity of the blockchain
print("Is blockchain valid?",
      my_blockchain.is_chain_valid())
```

```
Is blockchain valid? True
```

9. Demonstrating Tampering

```
# Change the data of an existing block
my_blockchain.chain[1].data = "Alice sends 1000 coins to Bob"

# Check the blockchain after modification
print("Is blockchain valid after tampering?",
      my_blockchain.is_chain_valid())
```

```
Is blockchain valid after tampering? False
```

Conclusion

The experiment successfully demonstrated the basic working of a blockchain using Python by creating blocks, linking them through hashes, and implementing simple Proof-of-Work mining. SHA-256 hashing was used to maintain data integrity, while blockchain validation was used to detect any modification or tampering. Thus, the experiment provided a basic understanding of block creation, hashing, mining, linking, and validation in blockchain technology.